Logistic Regression

Latreche Sara

Contents

1 Introduction to Logistic Regression

While simple linear classifiers like the Perceptron are effective for establishing a "hard" decision boundary, they are limited by their inability to express uncertainty. In real-world scenarios, we often need to know more than just a binary classification; we need to know the level of confidence in that prediction. **Logistic Regression** addresses this by evolving the linear framework into a probabilistic model. Instead of drawing a sharp line where a data point is either 100% on one side or 100% on the other, Logistic Regression models the **probability** that a given input belongs to a specific category.

Conceptually, this is achieved by passing the linear signal through a "squashing" function, known as the **Sigmoid function**. This transforms any numerical input into a value between 0 and 1, representing a probability. This approach is vital in high-stakes fields like **medical diagnosis** or **financial risk assessment**, where a prediction of "spam" or "fraud" is far more useful when accompanied by a percentage of certainty. Despite the complexity of the probabilities it produces, Logistic Regression remains a linear classifier at its core because the underlying threshold that separates classes remains a straight line—or a flat hyperplane in higher dimensions.

Modern applications of Logistic Regression frequently incorporate advanced techniques to maintain accuracy. regularization strategies such as **Ridge (L2)** and **Lasso (L1)** are used to keep the model parameters stable. By transitioning from the Perceptron's rigid logic to the probabilistic nature of Logistic Regression, we gain a more nuanced tool for interpreting data in the modern machine learning landscape.

Definition

Logistic Regression is a statistical model used in supervised learning for binary classification. Unlike linear regression which predicts continuous values, logistic regression predicts probabilities using the logistic (sigmoid) function.

1.1 Motivation

Suppose we want to predict whether a person has a disease ($y = 1$) or not ($y = 0$) based on their features (age, blood pressure, etc). Logistic regression gives the probability that $y = 1$ given the input features.

Note

The output of logistic regression is a probability, so the prediction is made by thresholding (usually at 0.5).

2 The Logistic Model

While it might seem reasonable at first to use linear regression to tackle classification tasks—where the output y is categorical—it turns out this approach can lead to unsatisfactory results. One key issue is that linear regression can predict values outside the valid range of classification labels (e.g., less than 0 or greater than 1), which doesn't make sense when $y \in \{0, 1\}$.

To address this, we introduce a different model: instead of predicting the outcome directly with a linear combination of the features, we pass that combination through a nonlinear function that restricts the output to the range $[0, 1]$.

The function we use is the **sigmoid function**, also known as the **logistic function**, defined as:

$$g(z) = \frac{1}{1 + e^{-z}}$$

We then define our hypothesis function as:

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

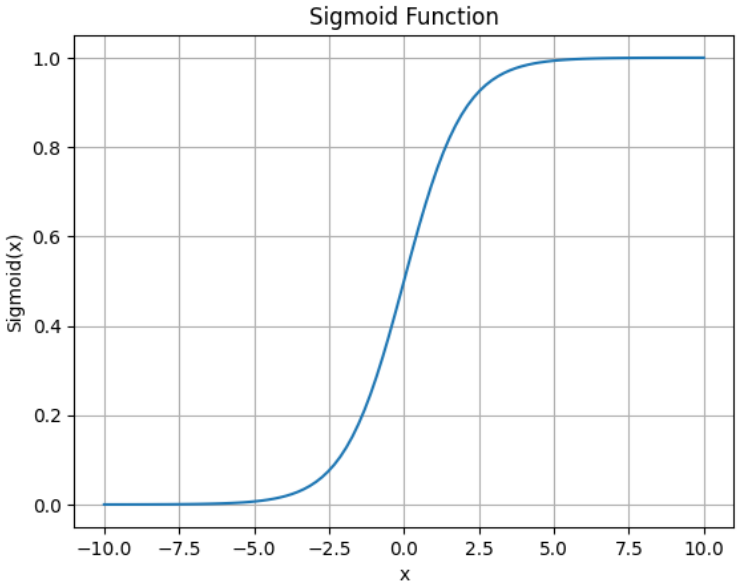
This formulation ensures that the output of $h_\theta(x)$ always lies strictly between 0 and 1, making it interpretable as the **probability** that the class label is 1, given the input features x. That is:

$$h_\theta(x) = P(y = 1 \mid x; \theta)$$

This is crucial in binary classification, where we want to decide between two classes. The logistic function behaves as follows:

- As $\theta^T x \to +\infty$, $h_\theta(x) \to 1$
- As $\theta^T x \to -\infty$, $h_\theta(x) \to 0$

A graph of $g(z)$ shows the familiar "S-shaped" curve.



This smooth curve transitions from 0 to 1, providing a natural mapping from real-valued inputs to probabilities. It's especially useful in cases where the decision boundary between the two classes isn't sharply defined.

To ensure compatibility with the intercept term, we continue to use the convention of augmenting the feature vector x with a bias term, setting $x_0 = 1$, so that the full linear expression becomes:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$$

An important analytical property of the sigmoid function is that its derivative has a simple and elegant form:

$$\frac{d}{dz} g(z) = g(z)(1 - g(z))$$

This derivative becomes particularly helpful when we derive the gradient of the cost function during model training.

While other activation functions could in theory be used to squeeze values between 0 and 1, the sigmoid function has several desirable mathematical properties that make it a standard choice, especially in the context of **generalized linear models (GLMs)** and probabilistic frameworks.

Maximum Likelihood Estimation of θ

With the logistic regression model in place, the next task is to determine how to optimally choose the parameters θ. Inspired by the probabilistic interpretation of logistic regression,

we can estimate θ using the principle of **maximum likelihood**.

Recall that we interpret the output of the model as a probability:

$$P(y = 1 \mid x; \theta) = h_\theta(x), \quad P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

These two cases can be unified into a single expression:

$$P(y \mid x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

Assuming that the training data consists of n i.i.d. examples $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$, the joint likelihood of the entire dataset is the product of the individual probabilities:

$$L(\theta) = \prod_{i=1}^{n} P(y^{(i)} \mid x^{(i)}; \theta) = \prod_{i=1}^{n} (h_\theta(x^{(i)}))^{y^{(i)}} (1 - h_\theta(x^{(i)}))^{1-y^{(i)}}$$

Rather than maximizing the likelihood directly, it's more convenient to work with the **log-likelihood**, which transforms the product into a sum:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log\big(1 - h_\theta(x^{(i)})\big) \right]$$

This log-likelihood function serves as our **objective function** for optimization. To find the optimal θ, we can apply techniques such as gradient ascent (or use gradient descent on the negative log-likelihood, also referred to as the logistic loss or cross-entropy loss).

The logistic regression model, therefore, emerges as both a principled probabilistic classifier and a tractable optimization problem.

Maximizing the Likelihood

To find the optimal parameters θ, we maximize the log-likelihood $\ell(\theta)$ using **gradient ascent**, since $\ell(\theta)$ is a concave function. Unlike gradient descent (which minimizes a function), gradient ascent updates the parameters in the direction of the positive gradient:

$$\theta := \theta + \alpha \nabla_\theta \ell(\theta),$$

where α is the learning rate.

Let us derive the gradient of the log-likelihood for a single training example (x, y).

Recall:

$$\ell(\theta) = y \log(h_\theta(x)) + (1 - y) \log(1 - h_\theta(x)), \quad \text{where} \quad h_\theta(x) = \sigma(\theta^T x).$$

We compute the partial derivative with respect to θ_j:

$$\frac{\partial}{\partial \theta_j}\ell(\theta) = y \cdot \frac{1}{h_\theta(x)} \cdot \frac{\partial h_\theta(x)}{\partial \theta_j} + (1-y) \cdot \frac{1}{1-h_\theta(x)} \cdot \left(-\frac{\partial h_\theta(x)}{\partial \theta_j}\right).$$

Factoring:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = \left(\frac{y}{h_\theta(x)} - \frac{1-y}{1-h_\theta(x)}\right) \cdot \frac{\partial h_\theta(x)}{\partial \theta_j}.$$

Now recall the derivative of the sigmoid function:

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1-\sigma(z)), \quad \text{and} \quad \frac{\partial h_\theta(x)}{\partial \theta_j} = h_\theta(x)(1-h_\theta(x)) \cdot x_j.$$

Putting it all together:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = \left(\frac{y}{h_\theta(x)} - \frac{1-y}{1-h_\theta(x)}\right) \cdot h_\theta(x)(1-h_\theta(x)) \cdot x_j.$$

This simplifies to:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = (y - h_\theta(x)) \cdot x_j.$$

Therefore, the **stochastic gradient ascent** update rule (using one training example at a time) is:

$$\theta_j := \theta_j + \alpha(y - h_\theta(x))x_j.$$

More compactly in vector form:

$$\theta := \theta + \alpha(y - h_\theta(x))x.$$

When using the full dataset (batch gradient ascent), we average the updates across all m examples:

$$\theta := \theta + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)}))x^{(i)}.$$

This completes the derivation of the logistic regression update rule using gradient ascent on the log-likelihood.

3 Gradient Ascent for Logistic Regression

Algorithm 1 Gradient Ascent for Logistic Regression

Require: Dataset $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, learning rate α, iterations T
Ensure: Optimized parameters θ

1: Initialize $\theta \leftarrow 0$
2: **for** each iteration $t = 1, \ldots, T$ **do**
3: Compute predictions: $h_\theta(x^{(i)}) = \sigma(\theta^T x^{(i)})$
4: Compute gradient of log-likelihood:

$$\nabla \ell(\theta) = \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x^{(i)}$$

5: Update parameters using gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla \ell(\theta)$$

6: **end for**

Cross-Entropy Loss: A Simple Example

Consider a binary classification problem where the true label y for a single data point is 1 (positive class). Thus, the true distribution is:

$$p = \begin{cases} 1 & \text{if } y = 1 \\ 0 & \text{if } y = 0 \end{cases}$$

Suppose the model predicts the probability of the positive class as $q = 0.9$. Then, the predicted probability of the negative class is $1 - q = 0.1$.
The cross-entropy loss for this example is given by

$$H(p, q) = -\big(y \log q + (1 - y) \log(1 - q)\big).$$

Since $y = 1$, this simplifies to

$$H(p, q) = -\log(0.9) \approx 0.105.$$

If instead the model predicts $q = 0.1$ (incorrect prediction), the cross-entropy loss becomes

$$H(p, q) = -\log(0.1) \approx 2.302.$$

Interpretation: When the model assigns a high probability to the true class, the cross-entropy loss is low, indicating a good prediction. Conversely, when the model assigns a low probability to the true class, the loss is high, signaling a poor prediction.

4 Example: Email Spam Detection

Example: Step-by-Step Gradient Ascent

Consider the following dataset with two email samples, where the feature vector includes a bias term (1) and the count of spam keywords:

$$x^{(1)} = \begin{bmatrix} 1 \\ 50 \end{bmatrix}, y^{(1)} = 1 \quad \text{and} \quad x^{(2)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y^{(2)} = 0$$

We want to optimize the parameter vector $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$ using gradient ascent.

Initialization: $\theta^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, learning rate $\alpha = 0.1$.

Iteration 1:

- **Compute predictions** using the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$h_{\theta^{(0)}}(x^{(1)}) = \sigma\left(\begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 50 \end{bmatrix}\right) = \sigma(0) = 0.5$$

$$h_{\theta^{(0)}}(x^{(2)}) = \sigma\left(\begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \sigma(0) = 0.5$$

- **Compute gradient** $\nabla \ell(\theta)$:

$$\begin{aligned} \nabla \ell(\theta^{(0)}) &= \sum_{i=1}^{2} (y^{(i)} - h_{\theta^{(0)}}(x^{(i)}))x^{(i)} \\ &= (1 - 0.5) \begin{bmatrix} 1 \\ 50 \end{bmatrix} + (0 - 0.5) \begin{bmatrix} 1 \\ 2 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 \\ 25 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 24 \end{bmatrix} \end{aligned}$$

- **Update parameters**:

$$\theta^{(1)} = \theta^{(0)} + \alpha \nabla \ell(\theta^{(0)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0.1 \begin{bmatrix} 0 \\ 24 \end{bmatrix} = \begin{bmatrix} 0 \\ 2.4 \end{bmatrix}$$

The update shows the vector θ moving in the direction of the gradient. The weight θ_1 increased to 2.4, effectively rotating the decision boundary to better classify the spam sample.

Interpretation of the First Update

After the first iteration, the updated parameter vector in $\mathbb{R}^2$ is:

$$\theta^{(1)} = \begin{bmatrix} 0 \\ 2.4 \end{bmatrix}$$

This result provides specific insights into the model's current state:

- The bias term (intercept) θ_0 remains 0, indicating no baseline shift in the classification threshold.
- The weight for the spam keyword feature, θ_1, has increased to 2.4.

Since $\theta_1 > 0$, the model has learned a positive correlation: as the keyword count increases, the probability of the "spam" class ($y = 1$) also increases.
To evaluate the current decision boundary, we compute the updated prediction for the second email $x^{(2)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$:

$$h_{\theta^{(1)}}(x^{(2)}) = \sigma\left(\begin{bmatrix} 0 & 2.4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}\right) = \sigma(4.8) \approx 0.991$$

Originally, the prediction for this email was 0.5. Now, the model predicts a 99.1% probability of it being spam. Since the true label is $y^{(2)} = 0$, this represents a high error. This phenomenon occurs because the large feature value in $x^{(1)}$ caused a significant weight update, causing the model to over-adjust toward the first sample—a behavior typical of early-stage learning.

In subsequent iterations, the gradient update will account for this error, eventually pulling the parameters back to a state that correctly balances the classification of both $x^{(1)}$ and $x^{(2)}$.

5 Decision Boundary

A logistic model defines a decision boundary where:

$$\theta^T x = 0 \quad \text{or} \quad h_\theta(x) = 0.5.$$

This boundary separates the feature space into two regions: predicted 0 or 1.

Note

If data is linearly separable, logistic regression can classify perfectly. For nonlinear cases, feature engineering or kernel methods are needed.

6 Multiclass Logistic Regression (Softmax)

To extend to $K > 2$ classes, we use the **softmax function**:

$$P(y = k \mid x) = \frac{e^{\theta_k^T x}}{\sum_{j=1}^{K} e^{\theta_j^T x}}.$$

This is called **Multinomial Logistic Regression**.

7 Conclusion

Logistic regression is a simple, powerful classification model with a solid statistical foundation. It is interpretable, fast to train, and effective for binary (and multiclass) problems.

Note

Despite its simplicity, logistic regression is widely used in industry for problems like credit scoring, spam filtering, and medical diagnosis.

References

[1] A. Ng. (2018). *Machine Learning Course*, Stanford University. https://www.coursera.org/learn/machine-learning